MediCare RAG Assistant

High-Level Design Document

RAG-Based Customer Support System

Healthcare & Insurance Domain

Version 1.0 | Internship Project

1. System Overview

1.1 Problem Definition

Healthcare and insurance organizations receive thousands of repetitive customer queries daily covering policy coverage, claims processing, premium billing, and plan comparisons. Traditional support channels are slow, expensive, and inconsistent. The MediCare RAG Assistant addresses this by providing an AI-driven self-service layer backed by authoritative policy documents.

Without RAG, a generic LLM would hallucinate policy-specific details or give outdated answers. By grounding the model in the organization's own PDFs, we ensure accuracy, traceability, and regulatory compliance.

1.2 Scope

- Ingest healthcare/insurance PDF knowledge bases into a vector store
- Answer natural language queries with cited, document-grounded responses
- Classify query intent across 7 categories automatically
- Route complex, sensitive, or low-confidence queries to human agents
- Provide a React-based web UI and a REST API
- Track and manage escalation tickets in a dedicated HITL dashboard

2. Architecture Diagram

The system follows a layered architecture with a clear separation between the user interface layer, the API gateway, the AI processing pipeline, and the data layer.

□ Architecture Layers (Top → Bottom)

[React Frontend (Vite + TailwindCSS)] ↕ HTTP/REST ↕ [FastAPI Backend — API Gateway & Orchestrator] ↕ [LangGraph Workflow Engine] input_node → retrieve_node → [Router] → generate_node / escalate_node ↕ ↕ [ChromaDB Vector Store] [HITL Service + Ticket Store] ↕ [OpenAI API (GPT-4o + text-embedding-3-small)]

2.1 Layer Descriptions

Layer	Description
React Frontend	Single-page application with 4 views: Chat, Knowledge Base, HITL Tickets, and System Status. Communicates via Axios REST calls.
FastAPI Backend	Python async API server. Handles request validation, file uploads, CORS, and routes to the LangGraph pipeline.
LangGraph Engine	Directed graph with 4 nodes and conditional routing. Manages state across nodes using a typed GraphState dictionary.
ChromaDB	Local persistent vector database storing document chunk embeddings. Uses cosine similarity for retrieval.
OpenAI API	Powers both the embedding model (text-embedding-3-small) and the language model (GPT-4o) for generation.
HITL Service	In-memory (extensible to DB) ticket store. Creates, manages, and resolves escalation tickets.

3. Component Descriptions

3.1 Document Loader

Implemented using PyMuPDF (fitz). The loader opens the uploaded PDF, iterates each page, and extracts plain text using get_text('text'). Pages with no content are skipped. The page number is preserved in metadata for source citation.

> **Why PyMuPDF?**
> PyMuPDF outperforms pypdf and pdfplumber for complex PDFs with tables and multi-column layouts. It handles scanned PDFs better and is significantly faster on large documents.

3.2 Chunking Strategy

Text is split using LangChain's RecursiveCharacterTextSplitter with:

- Chunk size: 800 characters (configurable via environment variable)
- Overlap: 150 characters (19% overlap preserves cross-boundary context)
- Separators: paragraph breaks → newlines → periods → spaces (natural boundary hierarchy)

The 800-character chunk size was chosen after evaluating the tradeoff between context richness (larger chunks) and retrieval precision (smaller chunks). Insurance policy documents contain dense, self-contained clauses that fit well within this range.

3.3 Embedding Model

OpenAI text-embedding-3-small generates 1536-dimensional dense vectors for each chunk. This model offers an optimal balance of quality and cost, with ~62% lower cost than text-embedding-ada-002 while achieving higher MTEB benchmark scores.

3.4 Vector Store — ChromaDB

ChromaDB is used as a local persistent vector store. The collection is configured with cosine distance metric (hnsw:space = cosine). Embeddings are batch-inserted in groups of 50 to respect OpenAI rate limits. ChromaDB's PersistentClient ensures data survives application restarts.

3.5 Retriever

On each query, the retriever embeds the user's message using the same embedding model and performs an approximate nearest-neighbor search via HNSW (Hierarchical Navigable Small World graphs). It returns the top-K chunks (default: 5) along with their cosine distances, which are converted to confidence scores (score = 1 − distance).

3.6 LLM — GPT-4o

GPT-4o is used for two tasks: (1) intent classification (temperature = 0, deterministic) and (2) answer generation (temperature = 0.3, slightly creative). The RAG system prompt instructs the model to answer only from provided context and maintain an empathetic, professional tone appropriate for healthcare support.

3.7 Graph Workflow Engine — LangGraph

LangGraph compiles a directed acyclic graph (with one conditional branch) into an executable pipeline. Each node is a pure Python function that takes the GraphState dict and returns an updated state. The compiled graph is cached as a module-level singleton for performance.

3.8 Routing Layer

The conditional router (should_escalate_router) evaluates three signals after retrieval:

1. Sensitive keyword match in the query (e.g., lawsuit, malpractice, emergency)
2. Confidence score below the configurable threshold (default: 0.65)
3. Zero relevant chunks retrieved (missing context)

If any condition is true, the graph routes to the escalate_node; otherwise to generate_node.

3.9 HITL Module

When escalation is triggered, the HITLService creates a structured ticket containing: ticket ID, session ID, original query, escalation reason, top-3 context snippets, and status. The current implementation uses an in-memory dictionary (designed for easy swap to PostgreSQL in production). The HITL dashboard allows human agents to view open tickets and submit resolutions.

4. Data Flow

4.1 Ingestion Pipeline

PDF → Page extraction (PyMuPDF) → Recursive text splitting → Batch embedding via OpenAI → ChromaDB upsert with metadata (source, page, chunk_index) → Response with chunk count to UI.

4.2 Query Lifecycle

Step	What Happens
1. HTTP Request	React sends POST /api/v1/chat with {session_id, message, conversation_history}
2. FastAPI Validation	Pydantic validates schema; checks that ChromaDB is non-empty
3. input_node	LLM classifies query into one of 7 intent categories
4. retrieve_node	Query is embedded; top-5 similar chunks fetched from ChromaDB
5. Router	Checks confidence, keywords, and context availability
6a. generate_node	Builds context-injected prompt; GPT-4o generates answer
6b. escalate_node	Creates HITL ticket; returns escalation message with ticket ID
7. Response	FastAPI serializes QueryResponse back to React
8. UI Render	MessageBubble renders markdown, intent badge, confidence bar, sources

5. Technology Choices

5.1 Why ChromaDB?

- Open-source, embedded, zero infrastructure overhead — no Pinecone/Weaviate account needed
- Native Python SDK with LangChain integration
- HNSW index provides sub-millisecond retrieval on tens of thousands of chunks
- PersistentClient supports on-disk durability without a separate server process
- Production path: swap for Pinecone or Weaviate without changing the retrieval interface

5.2 Why LangGraph?

- Provides explicit, inspectable control flow compared to LangChain's opaque chains
- State is typed with TypedDict — no hidden state or magic
- Conditional routing is a first-class primitive, not a workaround
- Supports async nodes, parallel branches, and checkpointing for production
- Graph is compiled once and reused, amortizing initialization cost

5.3 Why GPT-4o?

- State-of-the-art reasoning and instruction-following for complex healthcare queries
- 128K context window allows multi-document RAG without chunking artifacts
- Structured output support for intent classification (JSON mode available)
- Competitive pricing vs. GPT-4 Turbo with faster inference

5.4 Why FastAPI?

- Async by default — handles concurrent chat sessions without thread overhead
- Automatic OpenAPI documentation at /docs
- Pydantic v2 integration for robust schema validation
- CORS middleware makes React development seamless

6. Scalability Considerations

6.1 Handling Large Documents

- Batch embedding (50 chunks/batch) prevents OpenAI rate limit errors
- PyMuPDF processes PDFs in a streaming fashion, keeping memory stable
- For 1000+ page documents, HNSW index remains O(log n) query time
- Multiple PDFs can be ingested into the same collection; metadata preserves source identity

6.2 Increasing Query Load

- FastAPI's async event loop handles hundreds of concurrent requests per worker
- Multiple Uvicorn workers (--workers 4) for CPU-bound tasks
- LLM calls are the primary bottleneck — GPT-4o supports parallel requests
- ChromaDB can be replaced with hosted Pinecone for globally distributed retrieval

6.3 Latency Concerns

Component	Typical Latency
Embedding (query)	~50–100ms (OpenAI API)
ChromaDB retrieval	~2–10ms (local HNSW)
GPT-4o generation	~500–2000ms (depends on output length)
Total E2E	~600–2200ms
HITL escalation	~100ms (no LLM generation)

Optimization Opportunity

Intent classification and retrieval can run in parallel (async gather) to reduce total latency by ~30%. This is a planned optimization for v1.1.